

Лабораторная работа №13

**Программирование в командном процессоре ОС UNIX. Ветвления
и циклы**

Липатников Андрей

2026.10.08

В данном отчёте представлены результаты выполнения лабораторной работы по теме «Программирование в командном процессоре ОС UNIX. Ветвления и циклы».

Описаны логические управляющие конструкции, циклы, команда `getopts`, а также разработаны четыре командных файла и программа на языке Си.

Сделаны выводы о достижении поставленной цели.

Содержание

1 Введение	7
1.1 Актуальность	7
1.2 Цель работы	7
1.3 Задачи	7
1.4 Объект и предмет исследования	8
1.5 Методы исследования	8
1.6 Схема выполнения работы	8
2 Теоретическая часть	10
2.1 Команда getopts	10
2.2 Логические управляющие конструкции	10
2.3 Циклы	10
2.4 Команды true и false	11
2.5 while vs until	11
3 Материалы и методы	12
4 Выполнение лабораторной работы	13
4.1 Задание 1. search.sh — getopts + grep	13
4.2 Задание 2. Программа на C + \$?	15
4.3 Задание 3. mkfiles.sh — создание/удаление N файлов	16
4.4 Задание 4. backup_dir.sh — листинг и первый запуск	17
4.5 Задание 4. backup_dir.sh — проверка и режим -recent	19
5 Результаты	21
6 Обсуждение результатов	22
7 Заключение	23
Список литературы	24
А Приложение А. Ответы на контрольные вопросы	25
A.1 1. Предназначение команды getopts	25
A.2 2. Отношение метасимволов к генерации имён файлов	25
A.3 3. Операторы управления действиями	25
A.4 4. Операторы для прерывания цикла	26

A.5	5. Назначение true и false	26
A.6	6. Что означает строка if test -f man\$s/\$i.\$s?	26
A.7	7. Различия между while и until	26
В	Приложение Б. Листинги	27
B.1	Б.1. search.sh	27
B.2	Б.2. check.c	28
B.3	Б.3. check.sh	28
B.4	Б.4. mkfiles.sh	29
B.5	Б.5. backup_dir.sh	29
С	Приложение В. Листинг команд	31

Список иллюстраций

1.1	Блок-схема выполнения лабораторной работы	9
4.1	Скрипт search.sh	13
4.2	Запуск search.sh	14
4.3	Программа check.c	15
4.4	Компиляция и скрипт check.sh	16
4.5	Скрипт mkfiles.sh	16
4.6	Запуск mkfiles.sh	17
4.7	Скрипт backup_dir.sh	18
4.8	chmod +x и запуск backup_dir.sh /etc etc.tar.gz	19
4.9	Проверка архивов и запуск с -recent	19

Список таблиц

5.1 Сводка результатов	21
----------------------------------	----

1 Введение

1.1 Актуальность

Логические управляющие конструкции и циклы — основа любых сложных командных файлов в ОС UNIX/Linux. Команда `getopts` позволяет разбирать флаги командной строки, а `while`, `until`, `for`, `case` дают возможность управлять последовательностью выполнения. Навык написания таких скриптов обязателен для системного администратора и разработчика.

1.2 Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

1.3 Задачи

1. Написать командный файл с `getopts` и `grep` для поиска строк в файле.
2. Написать программу на C, возвращающую код в зависимости от числа, и скрипт, анализирующий \$?.
3. Написать командный файл для создания/удаления N файлов.

4. Написать командный файл для упаковки tar с фильтром по времени (find -mtime).
5. Ответить на контрольные вопросы.

1.4 Объект и предмет исследования

Объект — командная оболочка ОС UNIX.

Предмет — логические управляющие конструкции и циклы в командных файлах.

1.5 Методы исследования

Практическая работа в терминале, изучение man-справок, написание скриптов и программы на C, проверка результатов, фиксация скриншотами.

1.6 Схема выполнения работы

На рис. 1.1 представлена общая последовательность действий при выполнении работы.

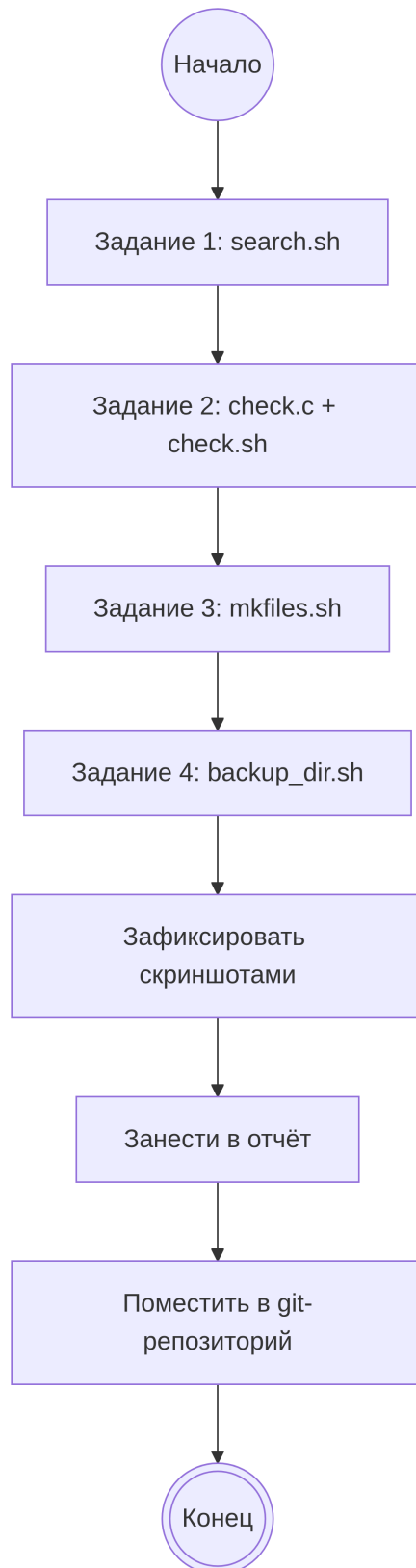


Рисунок 1.1: Блок-схема выполнения лабораторной работы

2 Теоретическая часть

2.1 Команда `getopts`

Синтаксис: `getopts option-string variable [arg ...]`.

Флаги с аргументами обозначаются двоеточием (`i:o:p:Cn`).

Специальные переменные: `OPTARG` (значение аргумента), `OPTIND` (индекс).

Принято включать `getopts` в цикл `while` и разбирать результаты через `case`.

2.2 Логические управляющие конструкции

- `if условие; then ... [elif ... then ...] [else ...] fi`
- `case имя in шаблон) команды;; ... esac`
- Логические связки `&&`, `||`
- `break`, `continue` — управление циклами

2.3 Циклы

- `for i in ...; do ... done`
- `while условие; do ... done`
- `until условие; do ... done`

2.4 Команды `true` и `false`

- `true` — всегда возвращает код 0 (истина).
- `false` — всегда возвращает ненулевой код (ложь). Используются для организации бесконечных циклов: `while true; do ... done`.

2.5 `while` vs `until`

- `while` выполняет тело, **пока** условие истинно.
- `until` выполняет тело, **пока** условие **ложно** (проверка инвертируется).

Более подробно про Unix см. в [1–4].

3 Материалы и методы

- **ОС:** Fedora 41 (Sway)
- **Терминал:** bash
- **Инструменты:** bash, grep, getopts, gcc, tar, find

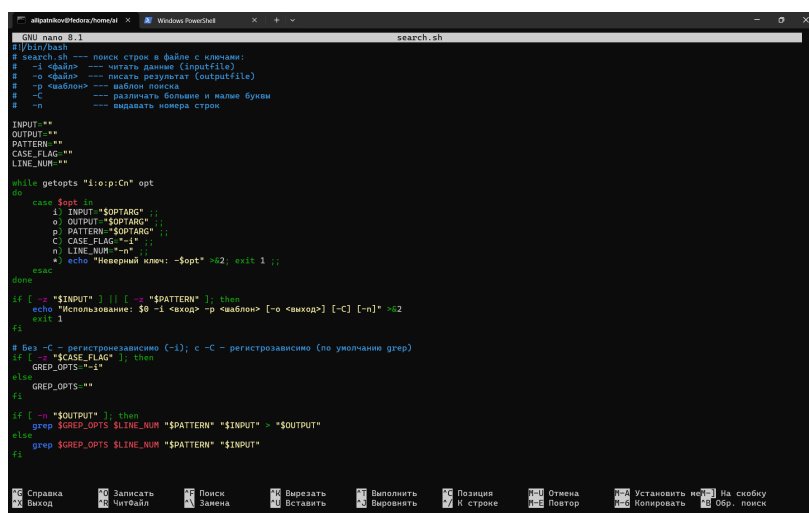
Рабочий каталог: `~/work/study/2025-2026/Операционные системы/os-intro/labs/lab13`.

Последовательность: написание четырёх скриптов и программы на С, проверка работы, фиксация **9 скриншотами**.

4 Выполнение лабораторной работы

4.1 Задание 1. search.sh — getopt + grep

Создан скрипт search.sh (рис. 4.1).



```
#!/bin/bash
# search.sh --- поиск строк в файле с клавиши:
# -i <файл> --- читать данные (inputfile)
# -o <файл> --- писать результат (outputfile)
# -p <шаблон> --- шаблон поиска
# -C --- различать большие и малые буквы
# -n --- выдавать номера строк

INPUT=""
OUTPUT=""
PATTERN=""
CASE_FLAG=""
LINE_NUM=""

while getopts "i:o:p:Cn" opt
do
    case $opt in
        i) INPUT="$OPTARG" ;;
        o) OUTPUT="$OPTARG" ;;
        p) PATTERN="$OPTARG" ;;
        C) CASE_FLAG="-C" ;;
        n) LINE_NUM="-n" ;;
        *) echo "Неверный ключ: -$opt" >&2; exit 1 ;;
    esac
done

if [ -z "$INPUT" ] || [ -z "$PATTERN" ]; then
    echo "Использование: $0 -i <вход> -p <шаблон> [-o <выход>] [-C] [-n]" >&2
    exit 1
fi

# Без -C - регистронезависимо (-i); с -C - регистрозависимо (по умолчанию grep)
if [ -z "$CASE_FLAG" ]; then
    GREP_OPTS="-i"
else
    GREP_OPTS=""
fi

if [ -n "$OUTPUT" ]; then
    grep $GREP_OPTS $LINE_NUM "$PATTERN" "$INPUT" > "$OUTPUT"
else
    grep $GREP_OPTS $LINE_NUM "$PATTERN" "$INPUT"
fi
```

Рисунок 4.1: Скрипт search.sh

```
#!/bin/bash

INPUT=""; OUTPUT=""; PATTERN=""; CASE_FLAG=""; LINE_NUM=""

while getopts "i:o:p:Cn" opt
do
    case $opt in
        i) INPUT="$OPTARG" ;;
        o) OUTPUT="$OPTARG" ;;
```

```

    p) PATTERN="$OPTARG" ;;
    C) CASE_FLAG="-i" ;;
    n) LINE_NUM="-n" ;;
    *) echo "Неверный ключ: -$opt" >&2; exit 1 ;;
esac
done
if [ -z "$INPUT" ] || [ -z "$PATTERN" ]; then
    echo "Использование: $0 -i <вход> -p <шаблон> [-o <выход>] [-C] [-n]" >&2
    exit 1
fi
if [ -z "$CASE_FLAG" ]; then GREP_OPTS="-i"; else GREP_OPTS=""; fi
if [ -n "$OUTPUT" ]; then
    grep $GREP_OPTS $LINE_NUM "$PATTERN" "$INPUT" > "$OUTPUT"
else
    grep $GREP_OPTS $LINE_NUM "$PATTERN" "$INPUT"
fi

```

Запуск и проверка (рис. 4.2).

```

aillipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/Lab13$ echo -e "Hello\nworld\nHELLO\nhello\nOther line" > test.txt
aillipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/Lab13$ cat test.txt
Hello
world
HELLO
hello
Other line
aillipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/Lab13$ ./search.sh -i test.txt
-p hello
Hello
HELLO
hello
aillipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/Lab13$ ./search.sh -i test.txt
-p hello -C
hello
aillipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/Lab13$ ./search.sh -i test.txt
-p hello -n
1:Hello
3:HELLO
4:hello
aillipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/Lab13$ ./search.sh -i test.txt
-p hello -o out.txt
aillipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/Lab13$ cat out.txt
Hello
HELLO
hello
aillipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/Lab13$ |

```

Рисунок 4.2: Запуск search.sh

- Ключи: -i (вход), -o (выход), -p (шаблон), -C (регистр), -n (номера строк).
- Без -C — регистронезависимо; с -C — регистрозависимо.

4.2 Задание 2. Программа на C + \$?

Создана программа check.c (рис. 4.3).

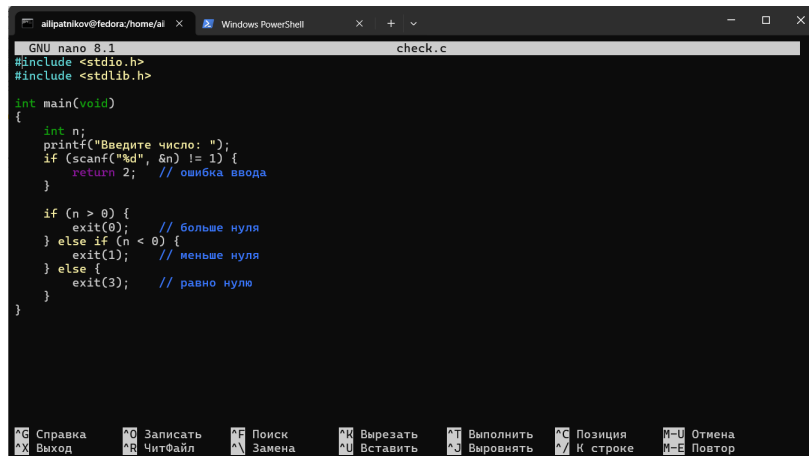


Рисунок 4.3: Программа check.c

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int n;

    printf("Введите число: ");
    if (scanf("%d", &n) != 1) return 2;
    if (n > 0) exit(0);
    else if (n < 0) exit(1);
    else exit(3);
}
```

Компиляция и запуск (рис. 4.4).

```

ailipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab13$ chmod +x check.sh
ailipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab13$ ./check.sh
Введите число: 5
Число больше нуля
ailipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab13$ ./check.sh
Введите число: -3
Число меньше нуля
ailipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab13$ ./check.sh
Введите число: 0
Число равно нулю
ailipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab13$ |

```

Рисунок 4.4: Компиляция и скрипт check.sh

- Коды возврата: 0 = > 0, 1 = < 0, 3 = == 0, 2 = ошибка ввода.
- Скрипт check.sh анализирует \$? через case.

4.3 Задание 3. mkfiles.sh — создание/удаление N файлов

Создан скрипт mkfiles.sh (рис. 4.5).

```

GNU nano 8.1 mkfiles.sh
#!/bin/bash
# mkfiles.sh --- создаёт/удаляет N файлов 1.tmp ... N.tmp
# Использование: $0 <N> <create|delete>

if [ $# -lt 2 ]; then
    echo "Использование: $0 <N> <create|delete>" >&2
    exit 1
fi

N="$1"
ACTION="$2"

case "$ACTION" in
    create)
        for i in $(seq 1 "$N"); do
            touch "${i}.tmp"
            echo "Создан: ${i}.tmp"
        done
        ;;
    delete)
        for i in $(seq 1 "$N"); do
            if [ -f "${i}.tmp" ]; then
                rm "${i}.tmp"
                echo "Удалён: ${i}.tmp"
            fi
        done
        ;;
    *)
        echo "Неверное действие: $ACTION" >&2
        exit 1
        ;;
esac

```

Рисунок 4.5: Скрипт mkfiles.sh

```

#!/bin/bash

if [ $# -lt 2 ]; then
    echo "Использование: $0 <N> <create|delete>" >&2

```

```

        exit 1
fi
N="$1"; ACTION="$2"
case "$ACTION" in
    create)
        for i in $(seq 1 "$N"); do touch "${i}.tmp"; done ;;
    delete)
        for i in $(seq 1 "$N"); do
            [ -f "${i}.tmp" ] && rm "${i}.tmp"
        done ;;
    *) echo "Неверное действие: $ACTION" >&2; exit 1 ;;
esac

```

Запуск и проверка (рис. 4.6).

```

ailipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab13$ chmod +x mkfiles.sh
ailipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab13$ ./mkfiles.sh 5 create
Создан: 1.tmp
Создан: 2.tmp
Создан: 3.tmp
Создан: 4.tmp
Создан: 5.tmp
ailipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab13$ ls *.tmp
1.tmp 2.tmp 3.tmp 4.tmp 5.tmp
ailipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab13$ ./mkfiles.sh 5 delete
Удалён: 1.tmp
Удалён: 2.tmp
Удалён: 3.tmp
Удалён: 4.tmp
Удалён: 5.tmp
ailipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab13$ ls *.tmp
ls: невозможно получить доступ к '*.tmp': Нет такого файла или каталога
ailipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab13$

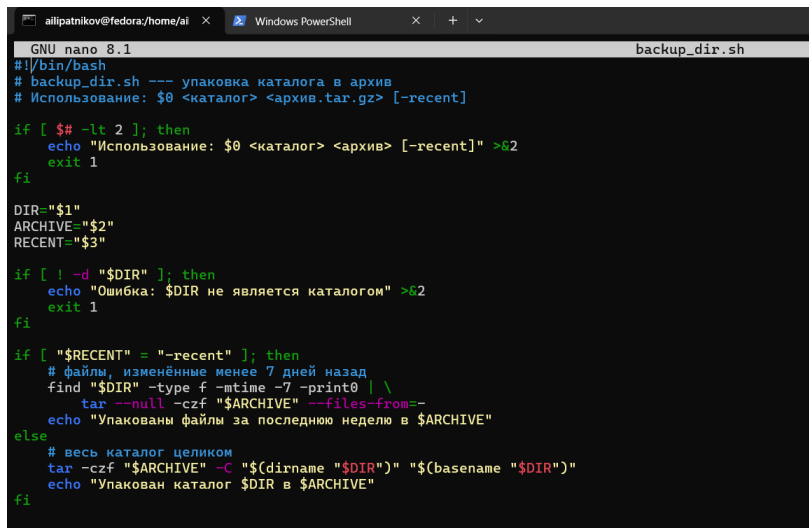
```

Рисунок 4.6: Запуск mkfiles.sh

- Создание: ./mkfiles.sh 5 create → 1.tmp ... 5.tmp.
- Удаление: ./mkfiles.sh 5 delete.

4.4 Задание 4. backup_dir.sh — листинг и первый запуск

Создан скрипт backup_dir.sh (рис. 4.7).

A screenshot of a terminal window with two tabs: 'allipatnikov@fedora/home/ai' and 'Windows PowerShell'. The active tab shows the 'backup_dir.sh' script being edited in the 'GNU nano 8.1' editor. The script is a shell script that takes three arguments: a directory, an archive name, and an optional flag '-recent'. It checks if the directory exists and if the flag is present. If the directory exists and the flag is not present, it archives the directory. If the flag is present, it archives only files modified in the last 7 days. The script uses 'find' and 'tar' commands. The terminal background is black, and the script text is in various colors (green, red, white) for syntax highlighting.

```
GNU nano 8.1 backup_dir.sh
#!/bin/bash
# backup_dir.sh --- упаковка каталога в архив
# Использование: $0 <каталог> <архив> [-recent]

if [ $# -lt 2 ]; then
    echo "Использование: $0 <каталог> <архив> [-recent]" >&2
    exit 1
fi

DIR="$1"
ARCHIVE="$2"
RECENT="$3"

if [ ! -d "$DIR" ]; then
    echo "Ошибка: $DIR не является каталогом" >&2
    exit 1
fi

if [ "$RECENT" = "-recent" ]; then
    # файлы, изменённые менее 7 дней назад
    find "$DIR" -type f -mtime -7 -print0 | \
        tar --null -czf "$ARCHIVE" --files-from=-
    echo "Упакованы файлы за последнюю неделю в $ARCHIVE"
else
    # весь каталог целиком
    tar -czf "$ARCHIVE" -C "$(dirname "$DIR")" "$(basename "$DIR")"
    echo "Упакован каталог $DIR в $ARCHIVE"
fi
```

Рисунок 4.7: Скрипт backup_dir.sh

```
#!/bin/bash
if [ $# -lt 2 ]; then
    echo "Использование: $0 <каталог> <архив> [-recent]" >&2
    exit 1
fi
DIR="$1"; ARCHIVE="$2"; RECENT="$3"
if [ ! -d "$DIR" ]; then
    echo "Ошибка: $DIR не является каталогом" >&2
    exit 1
fi
if [ "$RECENT" = "-recent" ]; then
    find "$DIR" -type f -mtime -7 -print0 | \
        tar --null -czf "$ARCHIVE" --files-from=-
else
    tar -czf "$ARCHIVE" -C "$(dirname "$DIR")" "$(basename "$DIR")"
fi
```

Скрипт сделан исполняемым и запущен в режиме полной упаковки каталога

/etc (рис. 4.8).

```
allipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab13$ nano backup_dir.sh
allipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab13$ chmod +x backup_dir.sh
allipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab13$ ./backup_dir.sh /etc etc.tar.gz
tar: etc/audit: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/credstore.encrypted: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/cups/cups.conf: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/cups/cups-classes.conf: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/cups/cups-files.conf: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/cups/cupsd.conf: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/cups/printers.conf: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/cups/subscriptions.conf: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/cups/cups-files.conf.default: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/cups/cupsd.conf.default: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/cups/snmp.conf.default: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/dhcrep: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/firewalld: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/fwupd/fwupd.conf: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/grub.d: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/gssproxy/2b-nfs-secctx.conf: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/gssproxy/99-network-fs-clients.conf: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/gssproxy/gssproxy.conf: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/ipsec.d: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/iscsi/iscsid.conf: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/lvm/archive: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/lvm/backup: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/lvm/cache: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/lvm/devices: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/ntfsables: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/opensvnm/client: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/opensvnm/server: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/pki/openssl: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/polkitt-1/localAuthority: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/polkitt-1/rules.d: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/ppp/chap-secrets: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/ppp/chap-secrets.sample: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/ppp/eaptls-client: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/ppp/eaptls-secrets: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/ppp/pap-secrets: Функция open завершилась с ошибкой: Отказано в доступе
tar: etc/security/opasswd: Функция open завершилась с ошибкой: Отказано в доступе
```

Рисунок 4.8: chmod +x и запуск backup_dir.sh /etc etc.tar.gz

- chmod +x backup_dir.sh — сделать исполняемым.
- ./backup_dir.sh /etc etc.tar.gz — упаковка всего каталога /etc.
- Без -recent: tar -czf etc.tar.gz -C / etc.

4.5 Задание 4. backup_dir.sh — проверка и режим -recent

Проверен первый архив и выполнен второй запуск — только файлы, изменённые менее 7 дней назад (рис. 4.9).

```
allipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab13$ ls -lh etc.tar.gz
-rw-r--r-- 1 allipatnikov allipatnikov 5,3M окт 9 23:14 etc.tar.gz
allipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab13$ tar -tzf etc.tar.gz | head
etc/
etc/favicon.png
etc/grub2-efi.cfg
etc/grub2.cfg
etc/issue
etc/issue.net
etc/opensc-x86_64.conf
etc/realvnc.conf
etc/ImageMagick-7/
etc/ImageMagick-7/colors.xml
allipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab13$ ./backup_dir.sh ~/work/study/2025-2026/Операционные системы/os-intro/
labs/lab13 recent.tar.gz -recent
tar: Удаленны начальные // из имен объектов
tar: Удаленны начальные // из целей жестких ссылок
tar: /home/allipatnikov/work/study/2025-2026/Операционные системы/os-intro/labs/lab13/recent.tar.gz: файл изменился во время чтения
Упомянутые файлы за последние недели в recent.tar.gz
allipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab13$ ls -lh recent.tar.gz
-rw-r--r-- 1 allipatnikov allipatnikov 11M окт 9 23:15 recent.tar.gz
allipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab13$ tar -tzf recent.tar.gz | head
home/allipatnikov/work/study/2025-2026/Операционные системы/os-intro/labs/lab13/presentation/image/lipatnikov.jpg
home/allipatnikov/work/study/2025-2026/Операционные системы/os-intro/labs/lab13/match.sh
home/allipatnikov/work/study/2025-2026/Операционные системы/os-intro/labs/lab13/test.txt
home/allipatnikov/work/study/2025-2026/Операционные системы/os-intro/labs/lab13/out.txt
home/allipatnikov/work/study/2025-2026/Операционные системы/os-intro/labs/lab13/check.c
home/allipatnikov/work/study/2025-2026/Операционные системы/os-intro/labs/lab13/check.sh
home/allipatnikov/work/study/2025-2026/Операционные системы/os-intro/labs/lab13/mfiles.sh
home/allipatnikov/work/study/2025-2026/Операционные системы/os-intro/labs/lab13/backup_dir.sh
home/allipatnikov/work/study/2025-2026/Операционные системы/os-intro/labs/lab13/etc.tar.gz
allipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab13$
```

Рисунок 4.9: Проверка архивов и запуск с -recent

- `ls -lh etc.tar.gz` — размер архива.
- `tar -tzf etc.tar.gz | head` — содержимое архива.
- `./backup_dir.sh ... recent.tar.gz -recent` — только файлы за неделю (`find -mtime -7`).
- `ls -lh recent.tar.gz` — размер второго архива.
- `tar -tzf recent.tar.gz | head` — содержимое второго архива.

5 Результаты

Таблица 5.1: Сводка результатов

Задание	Файл	Результат
1	search.sh	Поиск строк с ключами getopt + grep
2	check.c + check.sh	Программа на C + анализ \$?
3	mkfiles.sh	Создание/удаление N файлов
4	backup_dir.sh	Упаковка tar + фильтр по времени

- Все четыре задания методички выполнены.
- 9 снимков: по 2 на задания 1, 2 и 4, по 1 на задание 3 и листинги.

6 Обсуждение результатов

Все четыре задания выполнены. `getopts` корректно разбирает флаги с аргументами и без, `case` выбирает нужное действие. Программа на С возвращает разные коды завершения, скрипт анализирует их через `$?` и `case`. `mkfiles.sh` демонстрирует цикл `for` с `seq` и проверку `-f`. `backup_dir.sh` показывает связку `find + tar` через `--files-from=-`, что позволяет упаковать произвольный список файлов, найденных по критерию.

7 Заключение

- Цель работы достигнута: изучены основы программирования в оболочке UNIX.
- Освоена команда `getopts` и разбор флагов.
- Освоены логические конструкции (`if`, `case`) и циклы (`for`, `while`, `until`).
- Написаны и проверены четыре командных файла и программа на C.
- Освоена связка `find -mtime + tar`.

Список литературы

1. *Newham C.* Learning the bash Shell: Unix Shell Programming. — O'Reilly Media, 2005. — 354 p. — (In a Nutshell). — ISBN 0596009658. — URL: <http://www.amazon.com/Learning-bash-Shell-Programming-Nutshell/dp/0596009658>.
2. *Robbins A.* Bash Pocket Reference. — O'Reilly Media, 2016. — 156 p. — ISBN 978-1491941591.
3. *Zarrelli G.* Mastering Bash. — Packt Publishing, 2017. — 502 p. — ISBN 9781784396879.
4. *Таненбаум Э., Бос Х.* Современные операционные системы.. — 4-е изд. — СПб.: Питер, 2015.. — 1120 с. — (Классика Computer Science).

А Приложение А. Ответы на контрольные вопросы

А.1 1. Предназначение команды `getopts`

`getopts` — встроенная команда `bash` для разбора флагов командной строки. Она распознаёт опции, указанные в строке `option-string`, присваивает переменной букву опции, а `OPTARG` — её аргумент. Используется в цикле `while` с `case`.

А.2 2. Отношение метасимволов к генерации имён файлов

Метасимволы `*`, `?`, `[c1-c2]` используются оболочкой для `globbing` — подстановки имён файлов. Оболочка до запуска команды заменяет шаблон на список подходящих имён из текущего каталога. Это и есть генерация имён файлов.

А.3 3. Операторы управления действиями

- `if ... then ... [elif ... then ...] [else ...] fi` — условное выполнение.
- `case ... in ... esac` — выбор из нескольких вариантов.
- Логические связки `&&` (И), `||` (ИЛИ), `!` (НЕ).

- `break` — выход из цикла; `continue` — пропуск итерации.

A.4 4. Операторы для прерывания цикла

- `break` — завершает цикл полностью.
- `continue` — завершает текущую итерацию и переходит к следующей.

A.5 5. Назначение `true` и `false`

- `true` — всегда возвращает код 0 (истина).
- `false` — всегда возвращает ненулевой код (ложь). Используются для организации бесконечных циклов (`while true; do ... done`) и логических проверок.

A.6 6. Что означает строка `if test -f man$s/$i.$s?`

- `test -f` — проверка, что путь `man$s/$i.$s` — обычный файл.
- `$s` и `$i` — переменные оболочки, подставляются до выполнения.
- Если файл существует и является обычным — условие истинно, выполняется тело `if`.
- Иначе — переход к `else` (если есть) или к `fi`.

A.7 7. Различия между `while` и `until`

- `while` условие; `do ... done` — тело выполняется, **пока условие истинно**.
- `until` условие; `do ... done` — тело выполняется, **пока условие ложно** (проверка инвертируется). В остальном конструкции идентичны.

В Приложение Б. Листинги

В.1 Б.1. search.sh

```
#!/bin/bash
INPUT=""; OUTPUT=""; PATTERN=""; CASE_FLAG=""; LINE_NUM=""
while getopts "i:o:p:Cn" opt
do
    case $opt in
        i) INPUT="$OPTARG" ;;
        o) OUTPUT="$OPTARG" ;;
        p) PATTERN="$OPTARG" ;;
        C) CASE_FLAG="-i" ;;
        n) LINE_NUM="-n" ;;
        *) echo "Неверный ключ: -$opt" >&2; exit 1 ;;
    esac
done
if [ -z "$INPUT" ] || [ -z "$PATTERN" ]; then
    echo "Использование: $0 -i <вход> -p <шаблон> [-o <выход>] [-C] [-n]" >&2
    exit 1
fi
if [ -z "$CASE_FLAG" ]; then GREP_OPTS="-i"; else GREP_OPTS=""; fi
if [ -n "$OUTPUT" ]; then
```

```

    grep $GREP_OPTS $LINE_NUM "$PATTERN" "$INPUT" > "$OUTPUT"
else
    grep $GREP_OPTS $LINE_NUM "$PATTERN" "$INPUT"
fi

```

B.2 Б.2. check.c

```

#include <stdio.h>
#include <stdlib.h>
int main(void) {
    int n;
    printf("Введите число: ");
    if (scanf("%d", &n) != 1) return 2;
    if (n > 0) exit(0);
    else if (n < 0) exit(1);
    else exit(3);
}

```

B.3 Б.3. check.sh

```

#!/bin/bash
./check
rc=$?
case $rc in
    0) echo "Число больше нуля" ;;
    1) echo "Число меньше нуля" ;;

```

```

3) echo "Число равно нулю" ;;
2) echo "Ошибка ввода" ;;
*) echo "Неизвестный код возврата: $rc" ;;
esac

```

B.4 Б.4. mkfiles.sh

```

#!/bin/bash
if [ $# -lt 2 ]; then
    echo "Использование: $0 <N> <create|delete>" >&2
    exit 1
fi
N="$1"; ACTION="$2"
case "$ACTION" in
    create)
        for i in $(seq 1 "$N"); do touch "${i}.tmp"; done ;;
    delete)
        for i in $(seq 1 "$N"); do
            [ -f "${i}.tmp" ] && rm "${i}.tmp"
        done ;;
    *) echo "Неверное действие: $ACTION" >&2; exit 1 ;;
esac

```

B.5 Б.5. backup_dir.sh

```
#!/bin/bash
if [ $# -lt 2 ]; then
    echo "Использование: $0 <каталог> <архив> [-recent]" >&2
    exit 1
fi
DIR="$1"; ARCHIVE="$2"; RECENT="$3"
if [ ! -d "$DIR" ]; then
    echo "Ошибка: $DIR не является каталогом" >&2
    exit 1
fi
if [ "$RECENT" = "-recent" ]; then
    find "$DIR" -type f -mtime -7 -print0 | \
        tar --null -czf "$ARCHIVE" --files-from=-
else
    tar -czf "$ARCHIVE" -C "$(dirname "$DIR")" "$(basename "$DIR")"
fi
```

С Приложение В. Листинг команд

```
cd ~/work/study/2025-2026/"Операционные системы"/os-intro/labs/lab13

# Задание 1
nano search.sh
chmod +x search.sh
./search.sh -i test.txt -p hello
./search.sh -i test.txt -p hello -C
./search.sh -i test.txt -p hello -n
./search.sh -i test.txt -p hello -o out.txt

# Задание 2
nano check.c
gcc -o check check.c
nano check.sh
chmod +x check.sh
./check.sh

# Задание 3
nano mkfiles.sh
chmod +x mkfiles.sh
./mkfiles.sh 5 create
```

```
./mkfiles.sh 5 delete
```

```
# Задание 4
```

```
nano backup_dir.sh
```

```
chmod +x backup_dir.sh
```

```
./backup_dir.sh /etc etc.tar.gz
```

```
ls -lh etc.tar.gz
```

```
tar -tzf etc.tar.gz | head
```

```
./backup_dir.sh ~/work/study/2025-2026/"Операционные системы"/os-intro/labs/lab13 rec
```

```
ls -lh recent.tar.gz
```

```
tar -tzf recent.tar.gz | head
```